

React Native từ cơ bản đến nâng cao

— Tập 3: Nâng cao

New Architecture, native modules, performance, security, CI/CD, phát hành, monitoring

Expo SDK 57 · React Native 0.86.3 · React 19.2.3 · TypeScript 6.0.3

Bản dựng: 2026-09-28 — Markdown là nguồn chính (books/react-native/vol3-nang-cao/)

Kiểm tra dấu tiếng Việt: ă ằ ẳ ẵ ơ ư đ Ư Đ — Đường ướt, trắng khuyết, Ưu tiên

Mục lục

Chương 1 — New Architecture: JSI, Fabric, TurboModules, Codegen, Hermes

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 2 — Native modules với Expo Modules API

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 3 — Performance (hiệu năng)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 4 — Security (bảo mật)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 5 — CI/CD: GitHub Actions và EAS

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 6 — Phát hành lên App Store và Google Play

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 7 — Monitoring: Error Boundary, logging, báo lỗi từ xa

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 8 — App mẫu Tập 3: "Sổ Ghi Chú Bảo Mật"

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

GLOSSARY — Thuật ngữ React Native (Anh → Việt)

Chương 1 — New Architecture: JSI, Fabric, TurboModules, Codegen, Hermes

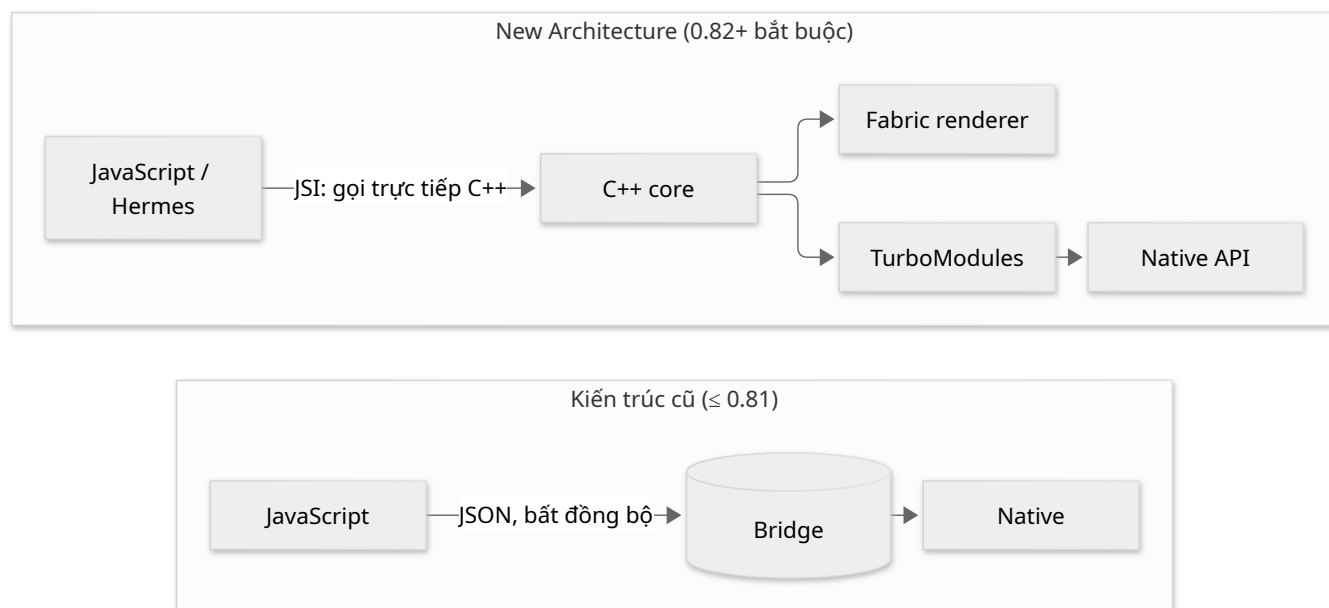
Mục tiêu

- Hiểu vì sao React Native bỏ "bridge" cũ và kiến trúc mới gồm những phần nào.
- Biết trạng thái hiện tại (2026-09): RN 0.82+ **chỉ** chạy New Architecture; Hermes V1 mặc định từ 0.84.
- Đọc được một **spec TurboModule** viết bằng TypeScript và hiểu Codegen làm gì.
- Kiểm tra engine đang chạy (Hermes) từ JavaScript.

Giải thích đơn giản

Hãy tưởng tượng app RN có hai "thế giới": **JavaScript** (code của bạn, chạy trong Hermes) và **native** (UIKit trên iOS, Android View). Chúng phải nói chuyện với nhau.

- **Kiến trúc cũ:** hai bên gửi thư qua một **bridge** (cây cầu): mọi lời gọi được chuyển thành JSON, xếp hàng, xử lý **bất đồng bộ**. An toàn nhưng chậm với cập nhật dày đặc, và không đọc được layout đồng bộ.
- **Kiến trúc mới:** JS giữ **tham chiếu trực tiếp** tới đối tượng C++ qua **JSI** (JavaScript Interface) → gọi hàm native **đồng bộ** được, không cần JSON.



Bốn mảnh ghép:

Thành phần	Là gì	Lợi ích
JSI	Lớp C++ cho JS gọi native trực tiếp	Không serialize JSON; gọi đồng bộ được
Fabric	Renderer (bộ vẽ giao diện) mới	Nhiều cây UI song song, đọc layout đồng bộ, hỗ trợ Suspense/Transitions
TurboModules	Native module thế hệ mới	Tải lười (lazy), gọi qua JSI, có kiểu
Codegen	Sinh code native từ spec TypeScript	JS và native luôn khớp kiểu

Và **Hermes**: JavaScript engine tối ưu cho mobile (biên dịch trước ra bytecode). Hermes V1 là mặc định từ RN 0.84.

Ví dụ

1. Spec của một TurboModule

examples/src/chapters/ch01/NativeDeviceInfoSpec.ts:

```
import type { TurboModule } from 'react-native';
import { TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  getDeviceName(): string; // hàm đồng bộ: gọi thẳng qua JSI, không qua "bridge" JSON
  getBatteryLevel(): Promise<number>;
}

export default TurboModuleRegistry.get<Spec>('NativeDeviceInfo');
```

Theo tài liệu "Turbo Native Modules": `TurboModuleRegistry.get<T>()` trả `null` nếu module không có; `getEnforcing<T>()` ném lỗi. Module này **không có trong Expo Go**, nên ta dùng `get` để app không crash.

Trong dự án RN CLI, bạn khai báo `codegenConfig` trong `package.json` (tên spec, thư mục `jsSrcsDir`, package Android); khi build, Codegen sinh interface C++/Objective-C++/Java từ spec. Với Expo, cách dễ hơn là **Expo Modules API** (Chương 2) — cũng chạy trên JSI.

2. Kiểm tra engine và phiên bản

examples/src/chapters/ch01/runtimeInfo.ts:

```

declare const global: { HermesInternal?: unknown };

export function getRuntimeInfo(): RuntimeInfo {
  const v = Platform.constants.reactNativeVersion;
  return {
    engine: global.HermesInternal ? 'Hermes' : 'khác',
    os: Platform.OS,
    reactNativeVersion: `${v.major}.${v.minor}.${v.patch}`,
    deviceName: NativeDeviceInfo?.getDeviceName() ?? '(không có module
NativeDeviceInfo – bình thường trong Expo Go)',
  };
}

```

Tài liệu Hermes của RN: biến toàn cục `HermesInternal` có mặt khi app chạy bằng Hermes.

Test — kết quả thật (2026-09-28)

```

PASS src/chapters/ch01/newArch.test.tsx
  ✓ Jest chạy trên Node (không phải Hermes); TurboModule không có → giá trị dự phòng
  ✓ có HermesInternal → báo Hermes (giống trên iPhone)
  ✓ hiển thị phiên bản React Native dạng major.minor.patch
Tests:      3 passed, 3 total

```

Phát hiện: trong Jest, `Platform.constants.reactNativeVersion` là bản giả và trả về `1000.0.0`. Lần đầu chúng tôi viết test mong đợi `0.86.x` và test **thất bại**. Trên iPhone với SDK 57 giá trị sẽ là `0.86.x` (**NOT RUN** trên máy thật).

Bằng chứng build: `npx expo export --platform ios` tạo được bundle **Hermes bytecode** (`.hbc`) cho cả 3 app — tức là mã của sách biên dịch được cho Hermes.

Đi sâu

Dòng thời gian (theo blog chính thức của React Native)

Phiên bản	Ngày	Mốc
0.76	2024-10-23	New Architecture là mặc định
0.82	2025-10-08	"the first React Native that runs entirely on the New Architecture" — <code>newArchEnabled=false</code> bị bỏ qua
0.84	2026-02-11	Hermes V1 mặc định; tiếp tục xóa code Legacy; iOS dùng binary biên dịch sẵn
0.85	2026-04-07	Shared Animation Backend (thử nghiệm); Jest preset tách thành <code>@react-native/jest-preset</code>
0.86	2026-06-11	Không breaking change; sửa edge-to-edge; repo chuyển sang tổ chức <code>react</code>
0.87	2026-08-11	Strict TypeScript API mặc định (chưa có trong Expo SDK 57)

Expo SDK 54 (RN 0.81) là bản cuối còn cho dùng Legacy Architecture. Tài liệu Expo: "Expo Go only supports the New Architecture."

Điều này có ý nghĩa gì với bạn?

- Thư viện cũ chỉ hỗ trợ bridge có thể **không chạy** trên RN 0.82+. Kiểm tra thư viện trước khi dùng (ví dụ `FlashList v2` ghi rõ "new architecture only").
- Bạn có thể gọi native **đồng bộ** (ví dụ `TextStats.stats(text)` ở Chương 2 trả kết quả ngay). Nhưng đừng làm việc nặng đồng bộ trên JS thread — UI sẽ đứng.
- React 19 features (Transitions, Suspense, automatic batching, `useLayoutEffect` đúng nghĩa) hoạt động đầy đủ nhờ Fabric.

Bridgeless

Blog 0.76 gọi là "Removing the Bridge": khởi động nhanh hơn, JS và native giao tiếp trực tiếp, báo lỗi tốt hơn. Từ 0.82, đây là chế độ duy nhất.

Lỗi và bẫy thường gặp

- Dùng `getEnforcing` cho module có thể **không tồn tại** (Expo Go, web) → crash ngay khi import.
- Tin số phiên bản trong Jest → là bản giả (`1000.0.0`).
- Cài thư viện native cũ (chưa hỗ trợ New Architecture) → lỗi build hoặc lỗi runtime "module not found".

- **Làm việc nặng trong hàm native đồng bộ** → khóa JS thread; dùng `AsyncFunction/Promise` cho việc lâu.
- **Tìm cách tắt New Architecture** trên SDK 57 → không thể; chỉ có thể dùng SDK ≤ 54 (không khuyến khích).

Tóm tắt

- New Architecture = JSI + Fabric + TurboModules + Codegen; bắt buộc từ RN 0.82.
- Hermes V1 mặc định từ 0.84; kiểm tra bằng `global.HermesInternal`.
- Spec TypeScript + Codegen giữ JS và native khớp kiểu; dùng `get()` khi module có thể vắng mặt.

Bài tập (có lời giải)

Bài 1. Vì sao `NativeDeviceInfoSpec.ts` dùng `TurboModuleRegistry.get` thay vì `getEnforcing`? Viết test chứng minh app không crash khi module vắng mặt.

▼ Lời giải

Vì app chạy trong **Expo Go** (không có module tự viết) và trong **Jest** (không có native). `get` trả `null`, code dùng `?.` và giá trị dự phòng. Test đã có:

```
it('Jest chạy trên Node (không phải Hermes); TurboModule không có → giá trị dự phòng', () => {
  const info = getRuntimeInfo();
  expect(info.engine).toBe('khác');
  expect(info.deviceName).toMatch(/không có module NativeDeviceInfo/);
});
```

Nếu đổi sang `getEnforcing`, chính dòng `import` sẽ ném lỗi — mọi test import file này đều đổ.

Bài 2. Điền bảng: với mỗi việc, nên gọi native **đồng bộ** hay **bất đồng bộ**? (a) đếm từ một đoạn văn ngắn; (b) đọc file 50 MB; (c) lấy tên thiết bị; (d) gọi mạng.

▼ Lời giải

Việc	Cách	Lý do
(a) đếm từ đoạn ngắn	Đồng bộ (Function)	Rất nhanh, cần kết quả ngay để hiển thị
(b) đọc file 50 MB	Bất đồng bộ (AsyncFunction)	Lâu → sẽ đứng UI nếu đồng bộ
(c) tên thiết bị	Đồng bộ hoặc hằng số (Constant)	Giá trị có sẵn, không đổi
(d) gọi mạng	Bất đồng bộ	Thời gian không đoán trước

Quy tắc: đồng bộ chỉ cho việc **dưới vài mili-giây**.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- The New Architecture is here (0.76): <https://github.com/facebook/react-native-website/blob/main/website/blog/2024-10-23-the-new-architecture-is-here.mdx>
- React Native 0.82 (New Architecture only): <https://github.com/facebook/react-native-website/blob/main/website/blog/2025-10-08-react-native-0.82.mdx>
- React Native 0.84 (Hermes V1): <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-02-11-react-native-0.84.mdx>
- React Native 0.86: <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-06-11-react-native-0.86.mdx>
- Turbo Native Modules: <https://github.com/facebook/react-native-website/blob/main/docs/turbo-native-modules.md>
- What is Codegen: <https://github.com/facebook/react-native-website/blob/main/docs/the-new-architecture/what-is-codegen.mdx>
- Hermes: <https://github.com/facebook/react-native-website/blob/main/docs/hermes.md>
- Expo — New Architecture guide: <https://github.com/expo/expo/blob/main/docs/pages/guides/new-architecture.mdx>

Chương 2 — Native modules với Expo Modules API

Mục tiêu

- Biết khi nào cần viết code native (Swift/Kotlin) và khi nào không.
- Tạo một **local Expo module** bằng `create-expo-module --local`.
- Viết API TypeScript an toàn: dùng native khi có, **dự phòng bằng JS** khi chạy trong Expo Go/web/test.
- Hiểu **development build** và cách chạy module native trên máy thật.

Giải thích đơn giản

Expo SDK đã có sẵn rất nhiều module (camera, vị trí, lưu trữ...). Chỉ viết native khi:

- Cần API của iOS/Android mà chưa có thư viện (ví dụ SDK riêng của ngân hàng, thiết bị Bluetooth đặc thù).
- Cần hiệu năng cao ở tầng native (xử lý ảnh, mã hóa lớn).

Expo Modules API cho bạn viết module bằng Swift và Kotlin với cú pháp ngắn gọn (DSL — ngôn ngữ khai báo): `Name`, `Constant`, `Function`, `AsyncFunction`, `Events`, `View`... Bên dưới, module chạy trên JSI của New Architecture.

Quan trọng: Expo Go chỉ chứa module có sẵn của Expo. Module bạn tự viết **không có** trong Expo Go → cần **development build** ("Expo Go của riêng bạn").

Ví dụ

1. Tạo module

Lệnh thật đã chạy (2026-09-28) trong `vol3-nang-cao/examples`:

```
CI=1 npx -y create-expo-module@latest modules/text-stats --local --barrel \
  --name TextStats --description "Dem tu va ky tu (vi du sach)" \
  --package dev.nobin.textstats -p apple android --features Function Constant
```

Output (trích):

- ✓ Downloaded module template from npm registry.
- ✓ Created the module from template files
- ✓ Generated barrel file (index.ts)
- ✓ Successfully created Expo module in modules/modules/text-stats

(Công cụ tạo thêm một cấp `modules/` thừa; chúng tôi chuyển thư mục về `modules/text-stats`. Template có license MIT của Expo — file `modules/text-stats/LICENSE` được giữ nguyên.)

Cấu trúc:

```
modules/text-stats/  
  expo-module.config.json      # khai báo module cho iOS ("apple") và Android  
  ios/TextStatsModule.swift    # code Swift  
  ios/TextStats.podspec  
  android/build.gradle  
  android/src/main/java/dev/nobin/textstats/TextStatsModule.kt  
  src/TextStatsModule.ts       # nối JS ↔ native  
  src/jsFallback.ts            # bản JavaScript dự phòng  
  index.ts                     # API công khai
```

2. Code native

Swift (`ios/TextStatsModule.swift`):

```
public class TextStatsModule: Module {  
  public func definition() -> ModuleDefinition {  
    Name("TextStats")  
  
    Constant("platform") {  
      "ios"  
    }  
  
    Function("stats") { (text: String) -> [String: Int] in  
      let words = text.split(whereSeparator: { $0.isWhitespace }).count  
      // unicodeScalars = code point, cùng cách đếm với Array.from(text) bên JS  
      return ["words": words, "characters": text.unicodeScalars.count]  
    }  
  }  
}
```

Kotlin (`android/.../TextStatsModule.kt`):

```

class TextStatsModule : Module() {
    override fun definition() = ModuleDefinition {
        Name("TextStats")

        Constant("platform") {
            "android"
        }

        Function("stats") { text: String ->
            val words = text.trim().split(Regex("\\s+")).filter { it.isNotEmpty() }.size
            mapOf("words" to words, "characters" to text.codePointCount(0, text.length))
        }
    }
}

```

NOT RUN: sandbox không có Xcode/Android SDK nên chưa biên dịch Swift/Kotlin. Cứ pháp theo đúng template do `create-expo-module` sinh ra và tài liệu "Tutorial: Creating a native module".

3. Cầu nối TypeScript có dự phòng

`src/TextStatsModule.ts`:

```

import { NativeModule, requireOptionalNativeModule } from 'expo';

declare class TextStatsNativeModule extends NativeModule<Record<string, never>> {
    platform: string;
    stats(text: string): TextStatsResult;
    reverse(text: string): string;
}

// requireOptionalNativeModule trả về null khi module native không có (ví dụ trong
// Expo Go),
// thay vì ném lỗi như requireNativeModule. Nhờ vậy app vẫn chạy trong Expo Go.
export default requireOptionalNativeModule<TextStatsNativeModule>('TextStats');

```

Template gốc dùng `requireNativeModule` (ném lỗi nếu thiếu). Sách đổi sang `requireOptionalNativeModule` (có trong `expo SDK 57`) để cùng một code chạy được cả trong Expo Go.

`index.ts`:

```

export function textStats(text: string): TextStatsResult & { source: StatsSource } {
    if (NativeTextStats) return { ...NativeTextStats.stats(text), source: 'native' };
    return { ...statsInJs(text), source: 'js' };
}

```

`src/jsFallback.ts`:

```
export function statsInJs(text: string): TextStatsResult {
  const words = text.trim() === '' ? 0 : text.trim().split(/\s+/).length;
  return { words, characters: Array.from(text).length };
}
```

4. Test cả hai nhánh

```
it('có module native (giả lập development build) → gọi native', async () => {
  jest.resetModules();
  const stats = jest.fn(() => ({ words: 42, characters: 99 }));
  jest.doMock('expo', () => ({
    ...jest.requireActual('expo'),
    requireOptionalNativeModule: () => ({ platform: 'ios', stats }),
  }));
  const mod = require('../../../../modules/text-stats');
  expect(mod.isNativeAvailable()).toBe(true);
  expect(mod.textStats('abc')).toEqual({ words: 42, characters: 99, source: 'native'
});
});
```

Kết quả (2026-09-28):

```
PASS src/chapters/ch02/nativeModule.test.tsx
✓ bản JS đếm từ và ký tự theo code point (emoji = 1)
✓ bài tập: reverseText không làm vỡ emoji (bản JS)
✓ không có module native (như Expo Go) → dùng JS
✓ có module native (giả lập development build) → gọi native
```

Trong app: **Cài đặt** → **Lab** → **"Ch.2 — Native module text-stats"**. Trong Expo Go sẽ hiện "Nguồn: JavaScript" (NOT RUN trên máy).

Đi sâu

Chạy module native thật (development build)

```
# Cách 1: build trên máy (cần Mac + Xcode cho iOS)
npx expo run:ios
# Cách 2: build trên cloud bằng EAS (không cần Mac), rồi cài lên iPhone
npx eas-cli@latest build --profile development --platform ios
```

Profile `development` trong `examples/eas.json` có `"developmentClient": true`. Khi đó cần cài thêm `expo-dev-client` (**chưa cài** trong dự án để app vẫn tối giản cho Expo Go). Thư mục `ios/` và `android/` được sinh ra (Continuous Native Generation) và bị `.gitignore` — không sửa tay.

Các khối của Expo Modules API

Khối	Dùng khi
<code>Constant("x") { ... }</code>	Giá trị không đổi (đọc như thuộc tính)
<code>Function("f") { ... }</code>	Hàm đồng bộ, nhanh
<code>AsyncFunction("f") { ... }</code>	Việc lâu → trả Promise
<code>Events("onChange") + sendEvent</code>	Native báo sự kiện lên JS
<code>View(...)</code>	Component giao diện native

Inline modules (thử nghiệm)

Tài liệu Expo nói **inline modules** (từ SDK 56, trạng thái *experimental*) cho phép đặt file Swift/Kotlin ngay trong thư mục app mà không cần package riêng. Sách dùng local module (ổn định hơn).

Mock mặc định cho module

Tài liệu "Mocking native calls" của Expo: đặt file mock trong thư mục `mocks` của module để jest-expo tự dùng khi test. Sách dùng cách đơn giản hơn: bản JS dự phòng +

```
jest.doMock('expo', ...).
```

Lỗi và bẫy thường gặp

- Dùng `requireNativeModule` rồi chạy trong Expo Go → lỗi "Cannot find native module".
- Sửa code native rồi chờ Fast Refresh → không có tác dụng; phải build lại app.
- Đếm ký tự khác nhau giữa iOS/Android/JS: Swift `String.count` đếm grapheme, Kotlin `length` đếm UTF-16, JS `length` đếm UTF-16. Chọn **một** quy ước (sách chọn code point) và test nó.
- Tách chuỗi bằng `split('')` → vỡ emoji (xem bài tập).
- Commit thư mục `ios/`, `android/` **sinh tự động** → xung đột khi nâng SDK.

Tóm tắt

- Chỉ viết native khi thật cần; Expo Modules API giúp viết Swift/Kotlin ngắn gọn.
- `requireOptionalNativeModule` + bản JS dự phòng = một code cho Expo Go và development build.
- Module tự viết cần development build (`npm expo run:ios` hoặc EAS Build).

Bài tập (có lời giải)

Bài 1. Thêm hàm `reverse(text)` vào module: Swift, Kotlin và bản JS dự phòng. Đảo chuỗi không được làm vỡ emoji (`'ab👉'` → `'👉ba'`).

▼ Lời giải

Swift: `Function("reverse") { (text: String) -> String in String(text.reversed()) }`

Kotlin:

```
Function("reverse") { text: String ->
    val cps = text.codePoints().toArray().reversedArray()
    String(cps, 0, cps.size)
}
```

JS (`src/jsFallback.ts`):

```
export function reverseInJs(text: string): string {
    return Array.from(text).reverse().join('');
}
```

Test:

```
expect(reverseText('ab👉')).toBe('👉ba');
expect('ab👉'.split('').reverse().join('')).not.toBe('👉ba'); // cách sai: tách
theo UTF-16
```

Lưu ý: Swift `reversed()` đảo theo grapheme (ký tự hiển thị), Kotlin đảo theo code point — với emoji ghép (ví dụ 👨👩) hai bên có thể khác nhau. Phần native **NOT RUN**.

Bài 2. Biến `stats` thành bất đồng bộ cho văn bản rất dài. Cần đổi những gì?

▼ Lời giải

1. Native: đổi `Function("stats")` thành `AsyncFunction("stats")` (Swift/Kotlin giữ nguyên thân hàm).
2. TypeScript: `stats(text: string): Promise<TextStatsResult>`.
3. `textStats` trở thành `async` và trả `Promise`; bản JS: `async function statsInJs(...)`.
4. Component gọi trong `useEffect` hoặc dùng `TanStack Query`.

(Lời giải tham khảo; code dự án giữ bản đồng bộ.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo Modules — Get started (`create-expo-module --local`): <https://github.com/expo/expo/blob/main/docs/pages/modules/get-started.mdx>
- Expo Modules — Tutorial: Creating a native module: <https://github.com/expo/expo/blob/main/docs/pages/modules/native-module-tutorial.mdx>
- Expo Modules — Module API reference: <https://github.com/expo/expo/blob/main/docs/pages/modules/module-api.mdx>
- Expo Modules — Inline modules tutorial (experimental): <https://github.com/expo/expo/blob/main/docs/pages/modules/inline-modules-tutorial.mdx>
- Expo Modules — Mocking native calls: <https://github.com/expo/expo/blob/main/docs/pages/modules/mocking.mdx>
- Expo — Development builds introduction: <https://github.com/expo/expo/blob/main/docs/pages/develop/development-builds/introduction.mdx>
- Gói npm `create-expo-module` (chạy trực tiếp bằng npx): <https://www.npmjs.com/package/create-expo-module>

Chương 3 — Performance (hiệu năng)

Mục tiêu

- Hiểu nguyên nhân chính làm app RN chậm: render thừa, danh sách dài, việc nặng trên JS thread.
- Đo bằng `<Profiler>` của React (cả trong test).
- Dùng `memo`, `useCallback`, `useMemo` đúng chỗ; hiểu **React Compiler** (bật trong app Tập 3).
- Dùng **FlashList v2** cho danh sách.

Giải thích đơn giản

App RN có (ít nhất) hai luồng quan trọng: **JS thread** (chạy React, logic) và **UI thread** (vẽ, cuộn, animation native). App "giật" khi một trong hai bị nghẽn quá ~16 ms (60 khung hình/giây).

Nguồn chậm phổ biến nhất trên JS thread: **render thừa** — component render lại dù dữ liệu của nó không đổi. Trong Angular, bạn dùng `OnPush` + signals. Trong React:

- `memo(Component)` : bỏ qua render nếu props **bằng nhau** (so sánh nông).
- `useCallback(fn, deps)` : giữ **cùng một hàm** giữa các lần render (để `memo` so sánh được).
- `useMemo(() => value, deps)` : giữ kết quả tính toán.
- **React Compiler**: tự động chèn các memo này lúc build.

Ví dụ

1. Đếm render bằng Profiler

`examples/src/chapters/ch03/RenderCountDemo.tsx` (trích):

```
function Row({ id, label, onPress, onRender }: RowProps) {
  return (
    <Profiler id={id} onRender={onRender}>
      <Pressable onPress={onPress}>
        <Text>{label}</Text>
      </Pressable>
    </Profiler>
  );
}

const MemoRow = memo(Row);

export function RenderCountDemo({ onRender = noop, inlineCallback = false }) {
  const [query, setQuery] = useState('');
  const [taps, setTaps] = useState(0);
  const handlePress = useCallback(() => setTaps((t) => t + 1), []); // hàm ổn định giữa các lần render

  return (
    <View style={{ padding: 16, gap: 8 }}>
      <TextInput accessibilityLabel="Gõ để làm cha render lại" value={query} onChangeText={setQuery} />
      <Row id="plain" label="Row thường" onPress={handlePress} onRender={onRender} />
      <MemoRow id="memo" label="Row có memo" onPress={inlineCallback ? () => setTaps((t) => t + 1) : handlePress} onRender={onRender} />
    </View>
  );
}
```

`<Profiler>` đặt **bên trong** Row: chỉ khi Row thật sự render thì `onRender` mới chạy. (Lần đầu chúng tôi đặt Profiler bên ngoài và test đếm sai: Profiler vẫn báo commit dù MemoRow không render.)

Test:

```
test('memo + useCallback: gõ vào ô tìm kiếm không làm MemoRow render lại', async () => {
  const counts: Record<string, number> = { plain: 0, memo: 0 };
  const onRender: ProfilerOnRenderCallback = (id) => { counts[id] += 1; };
  const user = userEvent.setup();
  await render(<RenderCountDemo onRender={onRender} />);
  const afterMount = { ...counts };
  await user.type(screen.getByLabelText('Gõ để làm cha render lại'), 'abc');
  expect(counts.plain - afterMount.plain).toBeGreaterThan(0);
  expect(counts.memo - afterMount.memo).toBe(0);
});
```

Kết quả (2026-09-28):

```
PASS src/chapters/ch03/performance.test.tsx
  ✓ memo + useCallback: gõ vào ô tìm kiếm không làm MemoRow render lại
  ✓ bài tập: hàm inline (tạo mới mỗi lần render) làm memo mất tác dụng
Tests:      2 passed, 2 total
```

2. FlashList v2 trong app

examples/src/app/(app)/index.tsx:

```
<FlashList
  data={visible}
  keyExtractor={(n) => n.id}
  renderItem={({ item }) => <NoteRow note={item} onOpen={open} />}
  ListEmptyComponent={<Text style={styles.empty}>{loaded ? 'Chưa có ghi chú.' : 'Đang tải...'}}
</Text>}
/>
```

README của FlashList 2.0.2: v2 "has been rebuilt from the ground up for RN's new architecture", **không cần** `estimatedItemSize`, là "JS-only solution", và **chỉ chạy trên New Architecture**. FlashList **tái sử dụng** (recycle) view khi cuộn thay vì tạo mới — nhanh hơn FlatList với danh sách dài.

3. React Compiler trong app Tập 3

examples/app.json:

```
"experiments": { "reactCompiler": true }
```

Tài liệu Expo: từ SDK 54, Babel đã tự cấu hình cho React Compiler — chỉ cần bật cờ này.

Kiểm chứng thật: chúng tôi chạy `npx expo export --platform ios --no-bytecode` cho app Tập 3 và tìm trong bundle: code của layout `(app)/_layout.tsx` đã được biến đổi thành dạng `cache (j[3]===Symbol.for("react.memo_cache_sentinel"))?(...)` — dấu hiệu của React Compiler. Bundle Tập 1 (không bật cờ) không có dạng này trong code app.

Nhưng trong Jest thì không: test ở trên cho thấy "Row thường" vẫn render lại khi gõ. Nếu Compiler chạy trong Jest, phần tử `<Row ... />` với props ổn định đã được cache và không render lại. Vì vậy test chạy **code chưa biên dịch bởi Compiler** — đó là lý do sách vẫn dạy `memo/useCallback` thủ công và test chúng. (Kết luận này suy ra từ hành vi quan sát được; chúng tôi chưa tìm thấy tài liệu nói rõ.)

Đi sâu

Danh sách: checklist

1. `keyExtractor` ổn định (id, không dùng index).
2. Component dòng bọc `memo`; callback bằng `useCallback` (hoặc để Compiler lo).
3. Không tạo object/array mới trong `renderItem` nếu truyền xuống dòng `memo`.
4. Ảnh: kích thước cố định; ảnh lớn dùng thumbnail.
5. Danh sách rất dài / nhiều loại dòng → FlashList.

Đo trên máy thật

- **React Native DevTools** (mở từ Dev Menu): tab Profiler của React.
- `useBenchmark` / `useFlatListBenchmark` (có trong exports của `FlashList`) để so sánh.
- Luôn đo trên **bản release** hoặc máy thật: bản dev chậm hơn nhiều vì kiểm tra thêm.

Công việc nặng

- Tính toán lớn: chia nhỏ, dùng `useDeferredValue` / `startTransition` (React 19, Fabric hỗ trợ), hoặc chuyển xuống native (Chương 2).
- Animation: Reanimated chạy trên UI thread (Tập 2 Chương 4).
- Khởi động app: Hermes biên dịch bytecode trước; tránh import thư viện lớn ở màn hình đầu.

Lỗi và bẫy thường gặp

- `memo` + **callback inline** → memo vô dụng (xem bài tập).
- `useMemo` **cho mọi thứ** → code rối, đôi khi chậm hơn. Đo trước, tối ưu sau.
- **Đo trong Jest rồi kết luận về app** → Jest không chạy React Compiler; chỉ dùng để khẳng định hành vi.
- **FlashList v2 + test**: file `@shopify/flash-list/jestSetup` của bản 2.0.2 thay `FlashList` bằng `RecyclerView` lấy từ gói — nhưng gói không export `RecyclerView` → lỗi "Element type is invalid". Chúng tôi gặp lỗi này (ErrorBoundary của app đã bắt và hiện nó!). Cách sửa trong `jest.setup.js`: chỉ giả lập các hàm đo layout.
- **Profiler bọc bên ngoài component memo** → đếm sai.

Tóm tắt

- Render thừa là thủ phạm số 1; đo bằng Profiler.
- `memo` + `useCallback` (hoặc React Compiler) + `FlashList` cho danh sách.
- React Compiler chạy khi build bằng Metro (đã kiểm chứng trong bundle), không chạy trong Jest.

Bài tập (có lời giải)

Bài 1. Chứng minh bằng test: nếu truyền **hàm inline** (`onPress={() => ...}`) cho `MemoRow`, nó sẽ render lại mỗi lần cha render.

▼ Lời giải

```
test('bài tập: hàm inline (tạo mới mỗi lần render) làm memo mất tác dụng', async () => {
  const counts: Record<string, number> = { plain: 0, memo: 0 };
  const user = userEvent.setup();
  await render(<RenderCountDemo inlineCallback onRender={(id) => (counts[id] += 1)} />);
  const before = counts.memo;
  await user.type(screen.getByLabelText('Gõ để làm cha render lại'), 'abc');
  expect(counts.memo - before).toBe(3); // render lại mỗi phím
});
```

Mỗi lần render, `() => ...` là **hàm mới** → `memo` thấy prop `onPress` khác → render lại.

Bài 2. Trong `NotesScreen`, tìm kiếm lọc trên mọi phím gõ. Với 10.000 ghi chú, làm sao để gõ vẫn mượt?

▼ Lời giải

Dùng `useDeferredValue` của React 19 để việc lọc có độ ưu tiên thấp hơn việc cập nhật ô input:

```
const [query, setQuery] = useState('');
const deferredQuery = useDeferredValue(query);
const visible = useMemo(() => filterNotes(notes, deferredQuery), [notes, deferredQuery]);
```

Hoặc debounce 200–300 ms (hook `useDebounceValue` ở Tập 1 Chương 2). Với dữ liệu rất lớn, lọc bằng SQL (`WHERE title LIKE ?`) thay vì trong JS. (Lời giải tham khảo; chưa đo trên máy thật.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo — React Compiler: <https://github.com/expo/expo/blob/main/docs/pages/guides/react-compiler.mdx>
- React Native — Optimizing FlatList Configuration: <https://github.com/facebook/react-native-website/blob/main/docs/optimizing-flatlist-configuration.md>
- React Native — Performance overview: <https://github.com/facebook/react-native-website/blob/main/docs/performance.md>
- Expo — FlashList (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/flash-list.mdx>
- FlashList repo (README v2): <https://github.com/Shopify/flash-list>
- React — Profiler (mã nguồn react.dev): <https://github.com/reactjs/react.dev/blob/main/src/content/reference/react/Profiler.md>

- React — useDeferredValue: <https://github.com/reactjs/react.dev/blob/main/src/content/reference/react/useDeferredValue.md>

Chương 4 — Security (bảo mật)

Mục tiêu

- Biết cái gì **không bao giờ** được đặt trong app (khóa bí mật, mật khẩu thật).
- Lưu dữ liệu nhạy cảm bằng **expo-secure-store** (Keychain/Keystore).
- Làm màn hình khóa bằng PIN: hash + salt, so sánh an toàn, khóa tạm sau nhiều lần sai.
- Kiểm tra dữ liệu từ bên ngoài (deep link) và bảo vệ route.

Giải thích đơn giản

Quy tắc số 1: **mọi thứ trong bundle JS đều có thể bị đọc**. App tải về máy người dùng; người có kỹ năng có thể giải nén và đọc code. Vì vậy:

- Khóa API "bí mật" (secret key) phải nằm ở **server**, không nằm trong app.
- Biến `EXPO_PUBLIC_*` được nhúng thẳng vào bundle. Tài liệu Expo cảnh báo: "Do not store sensitive info, such as private keys, in `EXPO_PUBLIC_*` variables. These variables will be visible in plain-text in your compiled application."

Quy tắc số 2: dữ liệu nhạy cảm trên máy → **Keychain (iOS) / Keystore (Android)**, qua expo-secure-store. Tài liệu React Native (trang Security) mô tả Async Storage là kho "unencrypted" (không mã hóa) và khuyên không dùng nó cho token, bí mật.

Dữ liệu	Nơi lưu
Cài đặt giao diện, cache công khai	AsyncStorage / SQLite
Token đăng nhập, hash PIN, khóa mã hóa	expo-secure-store
Khóa bí mật của server (API secret)	Không lưu trong app — để ở backend

Ví dụ

1. PIN: không lưu PIN, chỉ lưu hash(salt + PIN)

`examples/src/security/pin.ts` (trích):

```

    export async function hashPin(pin: string, salt: string, digest: Digest):
    Promise<string> {
        return digest(`${salt}:${pin}`);
    }

    // So sánh độ dài cố định (constant-time) để không lộ thông tin qua thời gian so sánh.
    export function safeEqual(a: string, b: string): boolean {
        if (a.length !== b.length) return false;
        let diff = 0;
        for (let i = 0; i < a.length; i++) diff |= a.charCodeAt(i) ^ b.charCodeAt(i);
        return diff === 0;
    }

    // 5 lần sai → khóa 30 giây, mỗi 5 lần sai tiếp theo nhân đôi thời gian.
    export function registerAttempt(state: LockState, success: boolean, now: number):
    LockState {
        if (success) return INITIAL_LOCK;
        const failedAttempts = state.failedAttempts + 1;
        if (failedAttempts % 5 !== 0) return { failedAttempts, lockedUntil: null };
        const level = failedAttempts / 5;
        return { failedAttempts, lockedUntil: now + 30_000 * 2 ** (level - 1) };
    }

```

`validatePinFormat` từ chối PIN yếu: không phải số, dài < 4 hoặc > 6, toàn số giống nhau (1111), dãy liên tiếp (1234, 3456, 9876).

2. Lưu vào SecureStore

`examples/src/security/secureStorage.ts` (trích):

```

const OPTIONS: SecureStore.SecureStoreOptions = {
    keychainAccessible: SecureStore.WHEN_UNLOCKED_THIS_DEVICE_ONLY, // không sao lưu
    sang máy khác
};

export const sha256: Digest = (text) =>
Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256, text);

export function newSalt(): string {
    return Array.from(Crypto.getRandomBytes(16), (b) => b.toString(16).padStart(2,
    '0')).join('');
}

```

- `expo-crypto: digestStringAsync` (SHA-256), `getRandomBytes` (số ngẫu nhiên an toàn).
- `WHEN_UNLOCKED_THIS_DEVICE_ONLY`: chỉ đọc được khi máy mở khóa, và không đi theo bản sao lưu.

Giới hạn của ví dụ: SHA-256 một vòng với PIN 4–6 số **không đủ** chống dò khi kẻ tấn công lấy được hash (chỉ 10^6 khả năng). Ở đây hash nằm trong Keychain nên khó lấy, và khóa tạm làm chậm việc đoán qua giao diện. App thật nên dùng khóa phần cứng/sinh trắc học (expo-local-authentication — Face ID **không có** trong Expo Go) hoặc hàm dẫn xuất khóa chậm (PBKDF2/Argon2) ở tầng native.

3. Store xác thực

examples/src/state/auth.ts (trích):

```
unlock: async (pin, now = Date.now()) => {
  const { lock } = get();
  if (isLocked(lock, now)) return false;
  const stored = await secureStorage.readPin();
  const ok = !!stored && safeEqual(await hashPin(pin, stored.salt, sha256),
stored.hash);
  const next = registerAttempt(lock, ok, now);
  await secureStorage.writeLock(next); // lưu số lần sai: tắt app mở lại cũng không
"reset" được
  set({ lock: next, status: ok ? 'unlocked' : 'locked' });
  return ok;
},
```

4. Protected routes + khóa khi xuống nền

examples/src/app/_layout.tsx:

```
const sub = AppState.addListener('change', (next) => {
  if (next === 'background') lockNow();
});
// ...
<Stack screenOptions={{ headerShown: false }}>
  <Stack.Protected guard={unlocked}>
    <Stack.Screen name="(app)" />
  </Stack.Protected>
  <Stack.Protected guard={!unlocked}>
    <Stack.Screen name="lock" />
  </Stack.Protected>
</Stack>
```

Tài liệu Expo Router: khi `guard` là `false`, người dùng không vào được màn hình đó và được chuyển về màn hình khả dụng đầu tiên; khi `guard` đổi từ `true` sang `false`, lịch sử của màn hình đó bị xóa.

5. Deep link: không tin dữ liệu bên ngoài

examples/src/chapters/ch04/deepLink.ts:

```

const PREFIX = /^rnbookvol3:\/\/(.*)$/;
const ID = /^[a-z0-9-]{1,40}$/;

export function parseDeepLink(raw: string): DeepLinkTarget {
  const m = PREFIX.exec(raw.trim());
  if (!m) return { screen: 'invalid', reason: /^[a-z][a-z0-9+.-]*:/i.test(raw) ?
'Scheme không được phép' : 'URL không hợp lệ' };
  const path = m[1].split(/[?#]/)[0];
  const parts = path.split('/').filter(Boolean);
  if (parts.length === 0) return { screen: 'home' };
  if (parts[0] === 'note' && parts.length === 2 && ID.test(parts[1])) return { screen:
'note', id: parts[1] };
  return { screen: 'invalid', reason: 'Đường dẫn không hỗ trợ' };
}

```

Phát hiện khi viết test: phiên bản đầu dùng `new URL(raw)`. Với `rnbookvol3://note/../../../../etc`, URL tự chuẩn hóa thành host `note` + path `/etc` → link được chấp nhận như id `etc`, dấu hiệu `..` bị che mất. Chúng tôi chuyển sang phân tích bằng regex + allowlist. Màn hình ghi chú cũng kiểm tra lại tham số bằng `parseNoteId` trước khi dùng.

Test — kết quả thật (2026-09-28)

```

PASS src/chapters/ch04/security.test.ts
  ✓ không lưu PIN gốc: chỉ có salt + hash trong SecureStore
  ✓ mở khóa đúng/sai và khóa tạm sau 5 lần sai
  ✓ init đọc lại trạng thái từ SecureStore (như mở lại app)
  ✓ parseDeepLink("rnbookvol3://note/abc-123")
  ✓ parseDeepLink("rnbookvol3://")
  ✓ parseDeepLink("https://evil.example/note/abc")
  ✓ parseDeepLink("rnbookvol3://note/../../../../etc")
  ✓ parseDeepLink("không phải url")
  ✓ bài tập: phát hiện bí mật trong EXPO_PUBLIC_*
PASS src/security/pin.test.ts
  (8 tests: validatePinFormat ×5, hash + salt, safeEqual, khóa tạm)

```

Trong test, `expo-secure-store` được thay bằng Map trong bộ nhớ (`src/test/mockSecureStore.ts`) và `expo-crypto` bằng SHA-256 của Node (`node:crypto`) — cùng thuật toán, cùng kết quả hex.

Đi sâu

Checklist bảo mật cho app RN (tóm tắt từ trang Security của RN + tài liệu Expo)

1. Không có bí mật trong bundle / `EXPO_PUBLIC_*`.
2. Token → SecureStore; không log token; không gửi token vào công cụ monitoring (Chương 7 có lọc).
3. Chỉ HTTPS (iOS App Transport Security chặn HTTP mặc định).

4. OAuth trên mobile: dùng Authorization Code + **PKCE** (trang Security của RN giải thích vì sao; `expo-auth-session` có tùy chọn `usePKCE`), không nhúng client secret.
5. Kiểm tra mọi input từ deep link, push notification, clipboard.
6. Khóa màn hình nhạy cảm khi app xuống nền.
7. Cân nhắc SSL pinning cho app tài chính (cần native/development build).

Vì sao không mã hóa luôn ghi chú?

Ghi chú của app mẫu nằm trong SQLite **không mã hóa**. `expo-sqlite` hỗ trợ SQLCipher, nhưng tài liệu SDK 57 ghi "SQLCipher is not supported on Expo Go". Muốn mã hóa thật: development build + SQLCipher, khóa lưu trong SecureStore. Sách ghi rõ giới hạn này thay vì giả vờ an toàn.

requireAuthentication của SecureStore

Tài liệu SDK 57: tùy chọn `requireAuthentication` "is not supported in Expo Go when biometric authentication is available" (thiếu khóa `NSFaceIDUsageDescription`). Dùng trong development build.

Lỗi và bẫy thường gặp

- **Đặt API secret trong `.env` với tiền tố `EXPO_PUBLIC_`** → lộ ra bundle.
- **Lưu token trong `AsyncStorage` / `Zustand persist`** → không mã hóa.
- **So sánh hash bằng `===`** trong code nhạy cảm → nên dùng so sánh thời gian cố định.
- **Reset số lần sai khi khởi động lại app** → kẻ tấn công chỉ cần tắt/mở app. Lưu `LockState` vào SecureStore.
- **Tin `new URL()` để kiểm tra đường dẫn** → chuẩn hóa `..` che mất dấu hiệu tấn công.
- **Log dữ liệu nhạy cảm** vào breadcrumbs → logger của Chương 7 tự thay bằng `[ẩn]`.

Tóm tắt

- Không có bí mật trong app; dữ liệu nhạy cảm vào SecureStore.
- PIN: hash + salt, so sánh an toàn, khóa tạm lưu bền vững, khóa khi xuống nền.
- Protected routes + kiểm tra deep link = không vào được màn hình khi chưa mở khóa.

Bài tập (có lời giải)

Bài 1. Viết `findSuspiciousPublicEnv(env)` trả về tên các biến `EXPO_PUBLIC_*` có vẻ chứa bí mật (tên chứa `secret`, `password`, `private_key`, hoặc giá trị bắt đầu bằng `sk_live_` / `sk_test_`).

▼ Lời giải

```
examples/src/chapters/ch04/publicEnv.ts:
```

```
const SECRET_PATTERNS = [/secret/i, /private[_-]?key/i, /password/i,
/^sk_(live|test)_/i];

export function findSuspiciousPublicEnv(env: Record<string, string | undefined>):
string[] {
  return Object.entries(env)
    .filter(([k]) => k.startsWith('EXPO_PUBLIC_'))
    .filter(([k, v]) => SECRET_PATTERNS.some((re) => re.test(k) || re.test(v ??
    '')))
    .map(([k]) => k);
}
```

Chạy hàm này trong CI (Chương 5) với `process.env` trước khi build để chặn lỗi sớm.

Bài 2. Tắt app rồi mở lại có "xóa" được số lần nhập sai không? Viết test.

▼ Lời giải

Không — `LockState` được ghi vào `SecureStore` sau mỗi lần thử, và `init()` đọc lại. Test "init đọc lại trạng thái từ `SecureStore` (như mở lại app)" đặt `status: 'loading'` rồi gọi `init()` và thấy `status === 'locked'`; tương tự, `lock` được đọc lại từ `pin.lock`.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React Native — Security: <https://github.com/facebook/react-native-website/blob/main/docs/security.md>
- Expo — Environment variables: <https://github.com/expo/expo/blob/main/docs/pages/guides/environment-variables.mdx>
- Expo — SecureStore (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/securestore.mdx>
- Expo — Crypto (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/crypto.mdx>
- Expo — SQLite (SQLCipher và Expo Go): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/sqlite.mdx>
- Expo — LocalAuthentication (Face ID và Expo Go): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/local-authentication.mdx>
- Expo — AuthRequest types (`usePKCE`): <https://github.com/expo/expo/blob/main/packages/expo-auth-session/src/AuthRequest.types.ts>
- React Native — Networking (App Transport Security): <https://github.com/facebook/react-native-website/blob/main/docs/network.md>

- Expo Router — Protected routes: <https://github.com/expo/expo/blob/main/docs/pages/router/advanced/protected.md>

Chương 5 — CI/CD: GitHub Actions và EAS

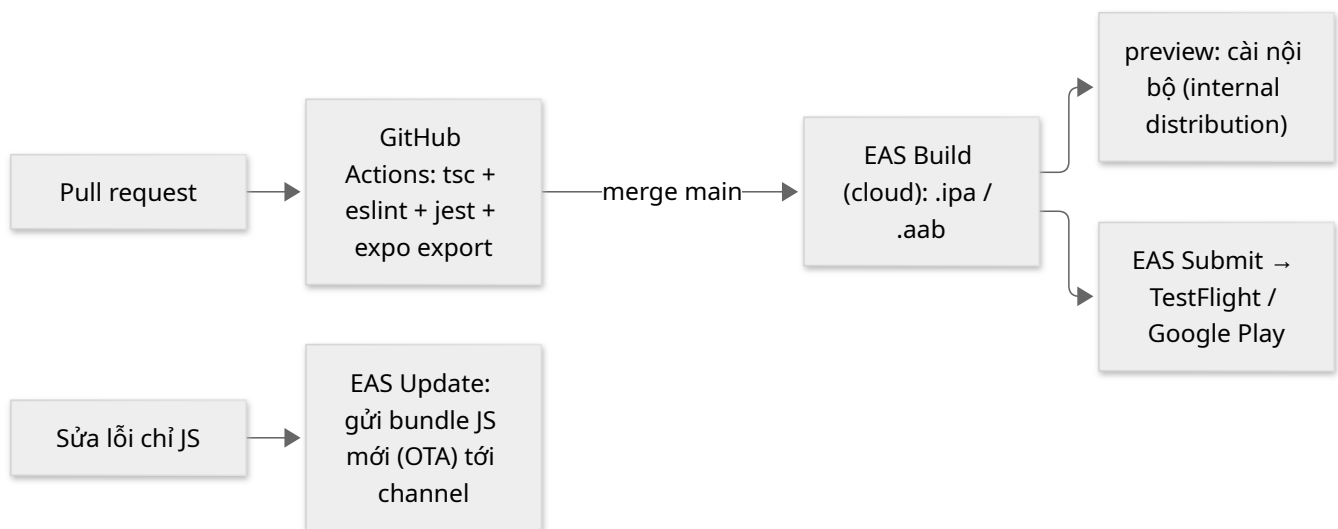
Mục tiêu

- Tự động chạy type-check, lint, test, export mỗi khi có pull request (CI).
- Hiểu **EAS Build** (build trên cloud), **EAS Submit** (gửi lên store), **EAS Update** (cập nhật OTA).
- Cấu hình `eas.json` với 3 profile: development, preview, production.
- "Kiểm tra cấu hình như code": test `app.json` và `eas.json` trước khi build.

Giải thích đơn giản

- **CI (Continuous Integration)**: mỗi PR, máy chủ tự chạy kiểm tra. Giống pipeline `ng lint && ng test && ng build` của bạn.
- **CD (Continuous Delivery)**: tự tạo bản build và gửi cho tester / store.

Với Expo:



Không cần Mac để build iOS: tài liệu EAS Build viết "iOS builds run on macOS runners hosted in Expo's macOS cloud" (image mặc định cho SDK 57: `macos-tahoe-26.5-xcode-26.6`, theo trang Build server infrastructure). Cần tài khoản Expo và Apple Developer để cài lên máy thật / gửi store.

Ví dụ

1. Workflow kiểm tra cho bộ sách

`vol3-nang-cao/ci/book-checks.yml` (mẫu — copy vào `.github/workflows/` ở gốc repo để bật):

```

name: React Native book checks
on:
  pull_request:
    paths:
      - 'books/react-native/**'
  push:
    branches: [main]
    paths:
      - 'books/react-native/**'
jobs:
  check:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      volume: [vol1-co-ban, vol2-trung-cap, vol3-nang-cao]
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v6
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: books/react-native/${{ matrix.volume
}}/examples/package-lock.json
      - name: Install
        run: npm ci
        working-directory: books/react-native/${{ matrix.volume }}/examples
      - name: Type-check, lint, test, export
        run: bash books/react-native/scripts/check-all.sh ${{ matrix.volume }}

```

Sách **không** tự thêm file vào `.github/workflows/` (ngoài phạm vi thư mục của task). File YAML được kiểm tra cú pháp bằng Python `yaml.safe_load` trong `check-all.sh` ("yaml OK: 2 file"). Workflow **chưa chạy trên GitHub** (NOT RUN) — nhưng lệnh bên trong (`check-all.sh`) đã chạy thật trong sandbox.

Bài học khi viết `check-all.sh`: phiên bản đầu viết `npx expo export ... && echo "web bundle OK"`. Khi export lỗi, `set -e` **không** dừng script (lệnh lỗi nằm trong chuỗi `&&`), và script vẫn in "ALL CHECKS PASSED". Chúng tôi phát hiện khi export web của Tập 2 lỗi (thiếu cấu hình `.wasm`), rồi sửa bằng `if ! npx expo export ...; then ... exit 1; fi`.

2. eas.json

`examples/eas.json`:

```

{
  "cli": {
    "version": ">= 24.8.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": { "developmentClient": true, "distribution": "internal" },
    "preview": { "distribution": "internal", "channel": "preview" },
    "production": { "autoIncrement": true, "channel": "production" }
  },
  "submit": { "production": {} }
}

```

- `development`: app có dev tools (cần `expo-dev-client`), cài nội bộ.
- `preview`: giống production nhưng phân phối nội bộ cho tester.
- `production`: bản lên store; `autoIncrement` tăng `ios.buildNumber`/`android.versionCode` (khi `appVersionSource` là `remote`, EAS lưu số build trên server — theo tài liệu "App versions").
- `channel`: nhận EAS Update đúng kênh.

`eas-cli` mới nhất trên npm: **24.8.0** (2026-09-28).

3. Workflow build bằng EAS (mẫu)

`vol3-nang-cao/ci/eas-build.yml` dựa theo ví dụ trong tài liệu Expo "Building on CI" (`expo/expo-github-action@v8`, secret `EXPO_TOKEN`):

```

- name: Setup Expo and EAS
  uses: expo/expo-github-action@v8
  with:
    eas-version: latest
    token: ${ secrets.EXPO_TOKEN }
- run: npm ci
- run: eas build --platform ios --profile preview --non-interactive --no-wait

```

NOT RUN: cần tài khoản Expo, `EXPO_TOKEN`, và Apple Developer. Xem Blockers.

4. Test cấu hình (chạy được ngay trong CI)

`examples/src/chapters/ch05/configRules.ts` (trích):

```

export function checkAppConfig(c: ExpoConfigLike): string[] {
  const errors: string[] = [];
  if (!c.version || !SEMV.test(c.version)) errors.push('version phải có dạng
x.y.z');
  if (!c.ios?.bundleIdentifier || !REVERSE_DNS.test(c.ios.bundleIdentifier))
errors.push('ios.bundleIdentifier phải dạng reverse-DNS');
  if (!c.android?.package || !REVERSE_DNS.test(c.android.package))
errors.push('android.package phải dạng reverse-DNS');
  if (!c.scheme) errors.push('cần scheme cho deep link');
  if (/secret|password|private[_-]?key/i.test(JSON.stringify(c)))
errors.push('app.json có vẻ chứa bí mật');
  return errors;
}

```

```

import appJson from '../../../../app.json';
import easJson from '../../../../eas.json';

it('app.json của dự án hợp lệ', () => {
  expect(checkAppConfig(appJson.expo)).toEqual([]);
});

```

Kết quả (2026-09-28):

```

PASS src/chapters/ch05/config.test.ts
  ✓ app.json của dự án hợp lệ
  ✓ eas.json của dự án hợp lệ
  ✓ phát hiện cấu hình sai
Tests:      3 passed, 3 total

```

Đi sâu

EAS Update (OTA)

Sửa lỗi chỉ ở JavaScript/asset có thể gửi thẳng tới máy người dùng mà không cần duyệt lại trên store: `npx eas-cli@latest update --channel production --environment production --message "Sửa lỗi PIN"` (dạng lệnh theo tài liệu "EAS Update getting started": `eas update --channel [channel-name] --message "[message]" --environment [environment-name]`). Tài liệu Expo: lệnh `eas update:configure` thêm `runtimeVersion` và `updates.url` vào app config; build nhận update theo `channel`. **Thay đổi native** (thêm module native, đổi quyền) vẫn cần build mới — update phải cùng `runtimeVersion` với build.

Chiến lược nhánh gợi ý

Sự kiện	Việc tự động
Pull request	<code>check-all.sh</code> (tsc, eslint, jest, export)
Merge vào <code>main</code>	EAS Build profile <code>preview</code> → tester cài thử
Tạo tag <code>v1.2.0</code>	EAS Build <code>production</code> + EAS Submit
Hotfix chỉ JS	EAS Update channel <code>production</code>

Cache và tốc độ CI

- `actions/setup-node` VỚI `cache: npm + package-lock.json` → cài nhanh hơn.
- Ma trận theo tập: 3 job chạy song song, một tập lỗi không che tập khác (`fail-fast: false`).

Lỗi và bẫy thường gặp

- `cmd && echo OK` trong script có `set -e` → lỗi bị nuốt (chúng tôi đã gặp).
- `npx expo install` trên CI không có mạng tới `api.expo.dev` → đặt `EXPO_OFFLINE=1` (sandbox của sách phải làm vậy).
- Quên `package-lock.json` → `npm ci` lỗi; CI cài phiên bản khác máy dev.
- Để `EXPO_TOKEN` trong code → phải là GitHub Secret.
- Gửi EAS Update có thay đổi native → app crash vì JS gọi module chưa có trong bản build.

Tóm tắt

- CI: một script kiểm tra duy nhất, chạy giống nhau ở máy dev và GitHub Actions.
- EAS Build/Submit/Update cho build cloud, gửi store, cập nhật OTA.
- Kiểm tra `app.json/eas.json` bằng test để bắt lỗi cấu hình trước khi tốn 20 phút build.

Bài tập (có lời giải)

Bài 1. Viết `checkEasJson(eas)` báo lỗi khi: thiếu một trong 3 profile; `appVersionSource=remote` nhưng `production` không bật `autoIncrement`; `production` là `development` client.

▼ Lời giải

```
export function checkEasJson(e: EasJsonLike): string[] {
  const errors: string[] = [];
  for (const p of ['development', 'preview', 'production']) if (!e.build?.[p])
errors.push(`thiếu build profile "${p}"`);
  if (e.cli?.appVersionSource === 'remote' && !e.build?.production?.autoIncrement)
{
    errors.push('appVersionSource=remote nên bật production.autoIncrement');
  }
  if (e.build?.production?.developmentClient) errors.push('production không được
là development client');
  return errors;
}
```

Test "phát hiện cấu hình sai" kiểm tra đủ 4 lỗi với một cấu hình xấu.

Bài 2. Thêm bước vào `book-checks.yml` để chạy `findSuspiciousPublicEnv` (Chương 4) trên biến môi trường của CI. Viết thế nào?

▼ Lời giải

Cách đơn giản: một test Jest đọc `process.env` và CI chạy test như bình thường:

```
it('không có bí mật trong EXPO_PUBLIC_* của môi trường build', () => {
  expect(findSuspiciousPublicEnv(process.env)).toEqual([]);
});
```

Trên GitHub Actions, đặt biến public ở `env:` của job; bí mật thật để trong `secrets` và **không** đặt tiền tố `EXPO_PUBLIC_`. (Lời giải tham khảo, chưa thêm vào bộ test.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo — EAS Build introduction: <https://github.com/expo/expo/blob/main/docs/pages/build/introduction.mdx>
- Expo — Build server infrastructure: <https://github.com/expo/expo/blob/main/docs/pages/build-reference/infrastructure.mdx>
- Expo — Building on CI: <https://github.com/expo/expo/blob/main/docs/pages/build/building-on-ci.mdx>
- Expo — Configure EAS Build with eas.json: <https://github.com/expo/expo/blob/main/docs/pages/build/eas-json.mdx>
- Expo — App versions (appVersionSource, autoIncrement): <https://github.com/expo/expo/blob/main/docs/pages/build-reference/app-versions.mdx>

- Expo — EAS Update getting started: <https://github.com/expo/expo/blob/main/docs/pages/eas-update/getting-started.mdx>
- Expo — Runtime versions: <https://github.com/expo/expo/blob/main/docs/pages/eas-update/runtime-versions.mdx>
- expo-github-action: <https://github.com/expo/expo-github-action>
- Gói npm eas-cli: <https://www.npmjs.com/package/eas-cli>

Chương 6 — Phát hành lên App Store và Google Play

Mục tiêu

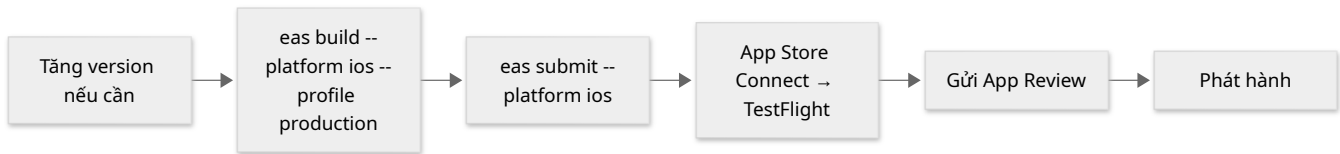
- Nắm quy trình từ code → bản build → TestFlight / Internal testing → phát hành.
- Hiểu **version** và **build number**, và cách EAS tự tăng số build.
- Chuẩn bị `app.json` cho store: tên, icon, bundle identifier/package, quyền, privacy manifest.
- Dùng `eas build` + `eas submit`.

Giải thích đơn giản

Hai con số luôn đi cùng nhau:

	Ví dụ	Ai thấy	Quy tắc
version (<code>expo.version</code>)	1.4.2	Người dùng trên store	Tăng theo semver khi có tính năng/sửa lỗi
build number (<code>ios.buildNumber</code> / <code>android.versionCode</code>)	57	Store dùng để phân biệt bản upload	Phải tăng mỗi lần upload

Quy trình (theo tài liệu EAS Submit):



Android tương tự: `eas build --platform android --profile production` (file `.aab`) → `eas submit --platform android` → track internal testing trên Google Play Console → production.

Ví dụ

1. app.json của app Tập 3

```
{
  "expo": {
    "name": "Sổ Ghi Chú Bảo Mật",
    "slug": "rn-book-vol3",
    "version": "1.0.0",
    "scheme": "rnbookvol3",
    "ios": { "supportsTablet": true, "bundleIdentifier":
"dev.nobin.rnbook.securenotes" },
    "android": { "package": "dev.nobin.rnbook.securenotes" },
    "experiments": { "reactCompiler": true }
  }
}
```

(Trích; file đầy đủ có icon, adaptive icon, plugins.) `bundleIdentifier/package` là **định danh vĩnh viễn** trên store — chọn kỹ trước lần upload đầu. Tên `dev.nobin...` là ví dụ; Nobin nên dùng domain của mình.

2. Tăng version bằng hàm thuần

`examples/src/chapters/ch06/version.ts`:

```
export function bumpVersion(version: string, kind: Bump): string {
  const m = /^(\d+)\.(\d+)\.(\d+)$/i.exec(version);
  if (!m) throw new Error(`Phiên bản không hợp lệ: ${version}`);
  let [major, minor, patch] = m.slice(1).map(Number);
  if (kind === 'major') [major, minor, patch] = [major + 1, 0, 0];
  else if (kind === 'minor') [minor, patch] = [minor + 1, 0];
  else patch += 1;
  return `${major}.${minor}.${patch}`;
}
```

Build number để EAS tự tăng: `"appVersionSource": "remote" + "autoIncrement": true` trong `eas.json` (Chương 5).

Kết quả test (2026-09-28):

```
PASS src/chapters/ch06/version.test.ts
  ✓ bumpVersion("1.4.2", "patch") → "1.4.3"
  ✓ bumpVersion("1.4.2", "minor") → "1.5.0"
  ✓ bumpVersion("1.4.2", "major") → "2.0.0"
  ✓ từ chối phiên bản sai định dạng
  ✓ bài tập: bumpFromCommits
Tests:      5 passed, 5 total
```

3. Lệnh build và submit (NOT RUN)

```
cd books/react-native/vol3-nang-cao/examples
npx eas-cli@latest login
npx eas-cli@latest build:configure           # liên kết project với tài khoản Expo
npx eas-cli@latest build --platform ios --profile production
npx eas-cli@latest submit --platform ios     # upload lên App Store Connect →
TestFlight
npx eas-cli@latest build --platform android --profile production
npx eas-cli@latest submit --platform android # lần đầu: tạo release ở track
internal testing
```

NOT RUN trong sandbox: cần tài khoản Expo, Apple Developer (trả phí) và Google Play Developer. Tài liệu EAS Submit: sau khi upload, build xuất hiện trong TestFlight "after processing (usually 10-15 minutes)"; `eas submit` chạy được trên macOS, Linux và Windows.

Đi sâu

Checklist trước khi gửi review

1. **Định danh:** `ios.bundleIdentifier`, `android.package` cố định.
2. **Icon & ảnh store:** đúng kích thước và định dạng (ví dụ trang "Store assets" của Expo ghi icon Google Play 512 × 512, PNG 32-bit; ảnh chụp màn hình 4–10 ảnh).
3. **Quyền:** chỉ xin quyền thật sự dùng; câu giải thích quyền rõ ràng (config plugin trong `app.json`, ví dụ `photosPermission` của `expo-image-picker`, `NSLocationWhenInUseUsageDescription` của `expo-location`).
4. **Privacy manifest (iOS):** nếu dùng API "required reason" (UserDefaults, file timestamp, disk space...), khai báo trong `expo.ios.privacyManifests` (tài liệu Expo "Privacy manifests").
5. **Nội dung store:** ảnh chụp màn hình, mô tả, chính sách quyền riêng tư (URL), thông tin liên hệ. Tài liệu Expo có trang "Store assets".
6. **Tài khoản demo** cho reviewer nếu app cần đăng nhập. App Tập 3 dùng PIN tự tạo — ghi chú điều này cho reviewer.
7. **Bản release test trên máy thật** (không chỉ bản dev).

Internal distribution vs TestFlight

- `preview` profile + internal distribution: cài thẳng lên máy đã đăng ký (ad hoc), nhanh cho team nhỏ.
- TestFlight: kênh test chính thức của Apple, mời tester qua email/link; bản build cần qua xử lý của Apple.

Android lần đầu

Tài liệu EAS Submit (Android): cần tạo app trên Google Play Console và tải **Google Service Account key** lên EAS. Lần submit đầu tạo release ở **internal testing track**; app ở trạng thái draft cho tới khi hoàn tất store listing. Nếu muốn, có thể tự upload lần đầu bằng tay (hướng dẫn "manual submission").

Lỗi và bẫy thường gặp

- **Upload trùng build number** → store từ chối. Để EAS `autoIncrement`.
- **Đổi `bundleIdentifier` sau khi đã phát hành** → thành app mới, mất người dùng cũ.
- **Xin quyền không dùng tới** → dễ bị từ chối review.
- **Quên `privacy manifest`** khi thư viện dùng API "required reason".
- **Gửi bản có dev menu / log nhạy cảm** → luôn build profile `production`.
- **Chỉ test trên Expo Go** → bản production khác (không có module Expo Go "thừa", có React Compiler, Hermes bytecode). Test bản `preview` trên máy thật trước.

Tóm tắt

- version (người dùng thấy) khác build number (phải tăng mỗi lần upload).
- `eas build + eas submit`; iOS qua TestFlight, Android qua internal testing.
- Chuẩn bị định danh, quyền, privacy manifest, nội dung store trước khi gửi review.

Bài tập (có lời giải)

Bài 1. Viết `bumpFromCommits(messages)` chọn loại tăng version từ Conventional Commits:

`feat!:/BREAKING CHANGE` → `major`; `feat:` → `minor`; `fix:` → `patch`; còn lại → `null`.

▼ Lời giải

```
export function bumpFromCommits(messages: string[]): Bump | null {
  if (messages.some((m) => /^[a-z]+(\.(.+\))?!:/test(m) || /BREAKING
CHANGE/.test(m))) return 'major';
  if (messages.some((m) => /^feat(\.(.+\))?!:/test(m))) return 'minor';
  if (messages.some((m) => /^fix(\.(.+\))?!:/test(m))) return 'patch';
  return null;
}
```

Kết hợp: `bumpVersion(appJson.expo.version, bumpFromCommits(commits) ?? 'patch')` trong một script release. Repo này cũng dùng Conventional Commits cho mọi commit.

Bài 2. Bạn sửa một lỗi hiển thị (chỉ JS) và một lỗi cần thêm quyền camera. Cái nào gửi bằng EAS Update được, cái nào cần build mới?

▼ Lời giải

- Lỗi hiển thị (chỉ JS/asset) → **EAS Update** được, cùng `runtimeVersion`.
- Thêm quyền camera → thay đổi native (Info.plist/AndroidManifest) → **build mới** + **submit** + **review**.

Quy tắc: bất cứ thứ gì làm thay đổi thư mục `ios/` / `android/` sinh ra (plugin, quyền, module native) đều cần build mới.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo — Submit to the Apple App Store: <https://github.com/expo/expo/blob/main/docs/pages/submit/ios.mdx>
- Expo — Submit to Google Play: <https://github.com/expo/expo/blob/main/docs/pages/submit/android.mdx>
- Expo — TestFlight: <https://github.com/expo/expo/blob/main/docs/pages/submit/testflight.mdx>
- Expo — App versions: <https://github.com/expo/expo/blob/main/docs/pages/build-reference/app-versions.mdx>
- Expo — Privacy manifests: <https://github.com/expo/expo/blob/main/docs/pages/guides/apple-privacy.mdx>
- Expo — Store assets: <https://github.com/expo/expo/blob/main/docs/pages/guides/store-assets.mdx>
- Expo — ImagePicker config plugin (photosPermission): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/imagepicker.mdx>

Chương 7 — Monitoring: Error Boundary, logging, báo lỗi từ xa

Mục tiêu

- Bắt lỗi render bằng **Error Boundary** để app không "trắng màn hình".
- Ghi **breadcrumbs** (dấu vết thao tác) và **báo lỗi** qua các transport, không lộ dữ liệu nhạy cảm.
- Biết cách nối với dịch vụ như **Sentry** (và giới hạn khi không có tài khoản).
- Test tất cả những điều trên.

Giải thích đơn giản

Khi app chạy trên máy người dùng, bạn không thấy console. Monitoring trả lời 3 câu hỏi: **Lỗi gì? Ở đâu? Người dùng đã làm gì trước đó?**

- **Error Boundary**: một component "lưới an toàn". Nếu component con ném lỗi khi render, nó hiện màn hình dự phòng ("Đã có lỗi xảy ra — Thử lại") thay vì làm sập cả cây.
- **Breadcrumbs**: danh sách ngắn các bước gần nhất ("mở màn hình", "tạo ghi chú"...), gửi kèm báo lỗi.
- **Transport**: nơi gửi báo lỗi — console khi dev, Sentry/Bugsnag khi production.

Trong Angular bạn có `ErrorHandler` toàn cục; React dùng Error Boundary theo từng nhánh cây.

Ví dụ

1. Logger với breadcrumbs và lọc dữ liệu nhạy cảm

`examples/src/monitoring/logger.ts` (trích):

```

const MAX_BREADCRUMBS = 30;
const SENSITIVE_KEY = /pin|password|token|secret/i;

export function createLogger(now: () => number = Date.now) {
  let crumbs: Breadcrumb[] = [];
  const transports: Transport[] = [];

  return {
    addBreadcrumb(level: Level, message: string, data?: Breadcrumb['data']) {
      // Không bao giờ ghi dữ liệu nhạy cảm vào log.
      const safe = data
        ? Object.fromEntries(Object.entries(data).map(([k, v]) => [k,
SENSITIVE_KEY.test(k) ? '[ẩn]' : v]))
        : undefined;
      crumbs = [...crumbs, { at: now(), level, message, data: safe }].slice(-
MAX_BREADCRUMBS);
    },
    reportError(error: unknown, context?: Record<string, string>) {
      const err = error instanceof Error ? error : new Error(String(error));
      const report: ErrorReport = { error: err, breadcrumbs: crumbs, context };
      for (const t of transports) {
        try {
          t(report);
        } catch {
          // transport lỗi không được làm hỏng app
        }
      }
      return report;
    },
    // ...
  };
}

```

App ghi breadcrumb ở các chỗ quan trọng: `auth.unlock`, `auth.unlock_failed` (kèm số lần thử, **không** kèm PIN), `note.create` (kèm độ dài, **không** kèm nội dung).

2. Error Boundary

`examples/src/monitoring/ErrorBoundary.tsx` (trích):

```

export class ErrorBoundary extends Component<Props, State> {
  state: State = { error: null };

  static getDerivedStateFromError(error: Error): State {
    return { error };
  }

  componentDidCatch(error: Error, info: ErrorInfo) {
    logger.reportError(error, { componentStack: (info.componentStack ?? '').slice(0, 500) });
    this.props.onError?.(error);
  }

  reset = () => this.setState({ error: null });

  render() {
    if (!this.state.error) return this.props.children;
    return (
      <View style={styles.box}>
        <Text accessibilityRole="alert" style={styles.title}>Đã có lỗi xảy ra 🙄</Text>
        <Text style={styles.msg}>{this.state.error.message}</Text>
        <Pressable accessibilityRole="button" onPress={this.reset} style={styles.btn}>
          <Text style={styles.btnText}>Thử lại</Text>
        </Pressable>
      </View>
    );
  }
}

```

Error Boundary **phải** là class component (React chưa có hook tương đương). Nó **không** bắt lỗi trong event handler, `setTimeout` hay Promise — những chỗ đó gọi `logger.reportError` bằng tay.

Layout gốc bọc toàn app: `<ErrorBoundary><AppShell /></ErrorBoundary>`, và gắn `consoleTransport`.

Chuyện thật khi viết sách: lúc test FlashList bị cấu hình sai (Chương 3), màn hình danh sách ném lỗi "Element type is invalid" — và test thấy màn hình "Đã có lỗi xảy ra 🙄" với đúng thông báo đó. Error Boundary đã làm đúng việc của nó.

3. Test

```
it('ErrorBoundary: bắt lỗi render, hiện màn hình dự phòng và báo cáo lỗi', async () => {
  const reports: ErrorReport[] = [];
  const remove = logger.addTransport((r) => reports.push(r));
  const consoleSpy = jest.spyOn(console, 'error').mockImplementation(() => {}); // React in
  lỗi ra console
  const user = userEvent.setup();
  await render(<Harness />);
  await user.press(screen.getByRole('button', { name: 'Gây lỗi' }));
  expect(screen.getByRole('alert')).toHaveTextContent(/Đã có lỗi xảy ra/);
  expect(screen.getByText('Nổ khi render!')).toBeOnTheScreen();
  expect(reports).toHaveLength(1);
  consoleSpy.mockRestore();
  remove();
});
```

Kết quả (2026-09-28):

```
PASS src/chapters/ch07/monitoring.test.tsx
  ✓ breadcrumbs: giữ 30 mục mới nhất và ẩn dữ liệu nhạy cảm
  ✓ reportError gửi tới mọi transport; transport lỗi không làm hỏng app
  ✓ ErrorBoundary: bắt lỗi render, hiện màn hình dự phòng và báo cáo lỗi
  ✓ bài tập: createDedupeTransport bỏ lỗi trùng trong 60 giây
```

Test toàn app cũng kiểm tra nút "💣 Thử lỗi (Error Boundary)" trong Cài đặt.

Hai bẫy gặp khi viết test:

1. Đặt `accessibilityRole="alert"` trên `<View>` → `getByRole('alert')` **không tìm thấy**, vì View chỉ là phần tử accessibility khi có `accessible={true}`. Chuyển role sang `<Text>` (Text mặc định accessible).
2. `info.componentStack` trong môi trường test không chứa tên component như mong đợi — test chỉ kiểm tra nó là chuỗi.

Đi sâu

Nối Sentry (chưa chạy — UNVERIFIED trong sandbox)

Tài liệu Expo "Using Sentry" khuyên dùng wizard:

```
npx @sentry/wizard@latest -i reactNative
```

Wizard cài gói, cấu hình Metro (để upload source map), và yêu cầu đăng nhập tài khoản Sentry. Phiên bản `@sentry/react-native` mà Expo SDK 57 khuyến nghị (bundledNativeModules.json): `~7.11.0`. Sau đó, bạn có thể nối logger của sách với Sentry bằng một transport:

```
import * as Sentry from '@sentry/react-native';

logger.addTransport(({ error, breadcrumbs, context }) => {
  Sentry.withScope((scope) => {
    breadcrumbs.forEach((b) => scope.addBreadcrumb({ message: b.message, level:
b.level === 'warn' ? 'warning' : b.level, data: b.data }));
    if (context) scope.setContext('app', context);
    Sentry.captureException(error);
  });
});
```

Đoạn code Sentry trên **chưa được cài và chạy** (không có tài khoản/DSN, và sách không muốn thêm phụ thuộc chỉ để minh họa) → **UNVERIFIED**. Kiểm tra API trong tài liệu Sentry trước khi dùng.

Expo Router cũng có ErrorBoundary theo route

Tài liệu Expo Router ("Error handling"): mỗi file route có thể `export function ErrorBoundary({ error, retry })` để bắt lỗi riêng của màn hình đó. Sách dùng một Error Boundary ở gốc cho đơn giản; app lớn nên dùng cả hai.

Đo gì trong production?

- Tỷ lệ phiên không crash (crash-free sessions).
- Lỗi JS theo phiên bản app / runtimeVersion (để biết EAS Update nào gây lỗi).
- Thời gian khởi động, màn hình chậm.
- **Không** thu thập nội dung ghi chú, PIN, token.

Lỗi và bẫy thường gặp

- **Nghĩ Error Boundary bắt mọi lỗi** → không bắt lỗi trong handler/Promise.
- **Log dữ liệu cá nhân** → vi phạm quyền riêng tư; lọc trước khi gửi (xem `SENSITIVE_KEY`).
- **Không có source map** → stack trace production khó đọc (bundle đã minify/Hermes bytecode).
- **Transport ném lỗi làm hỏng app** → bọc `try/catch` từng transport.
- **Bão lỗi**: một lỗi lặp lại hàng trăm lần mỗi phút → cần dedupe/sampling (bài tập).

Tóm tắt

- Error Boundary (class) ở gốc + ErrorBoundary theo route khi cần.
- Logger: breadcrumbs có giới hạn, lọc dữ liệu nhạy cảm, nhiều transport an toàn.
- Sentry nối qua transport; cần tài khoản và source map (chưa chạy trong sách).

Bài tập (có lời giải)

Bài 1. Viết `createDedupeTransport(inner, windowMs)`: không gửi lại cùng một lỗi (cùng `name` + `message`) trong `windowMs`.

▼ Lời giải

`examples/src/monitoring/dedupe.ts`:

```
export function createDedupeTransport(inner: Transport, windowMs: number, now: () => number = Date.now): Transport {
  const lastSent = new Map<string, number>();
  return (report) => {
    const key = `${report.error.name}:${report.error.message}`;
    const t = now();
    const prev = lastSent.get(key);
    if (prev !== undefined && t - prev < windowMs) return;
    lastSent.set(key, t);
    inner(report);
  };
}
```

Test: gửi A (t=0), A (t=30s, bị bỏ), B (t=30s), A (t=61s) → `inner` nhận `['A', 'B', 'A']`.

Bài 2. Thêm Error Boundary riêng cho màn hình ghi chú bằng API của Expo Router, để lỗi ở đó không thay cả app bằng màn hình lỗi.

▼ Lời giải

Trong `src/app/(app)/note/[id].tsx`:

```
import { type ErrorBoundaryProps } from 'expo-router';

export function ErrorBoundary({ error, retry }: ErrorBoundaryProps) {
  return (
    <View style={{ padding: 24, gap: 12 }}>
      <Text accessibilityRole="alert">Không mở được ghi chú: {error.message}</Text>
      <AppButton title="Thử lại" onPress={retry} />
    </View>
  );
}
```

Theo tài liệu Expo Router, route export `ErrorBoundary` sẽ được bọc bởi một React Error Boundary. (Lời giải tham khảo; chưa thêm vào app mẫu.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React — Catching rendering errors with an Error Boundary (mã nguồn react.dev): <https://github.com/reactjs/react.dev/blob/main/src/content/reference/react/Component.md>
- Expo Router — Error handling: <https://github.com/expo/expo/blob/main/docs/pages/router/error-handling.mdx>
- Expo — Using Sentry: <https://github.com/expo/expo/blob/main/docs/pages/guides/using-sentry.mdx>
- React Native — Security (không gửi token vào công cụ monitoring): <https://github.com/facebook/react-native-website/blob/main/docs/security.md>
- Sentry React Native repo: <https://github.com/getsentry/sentry-react-native>

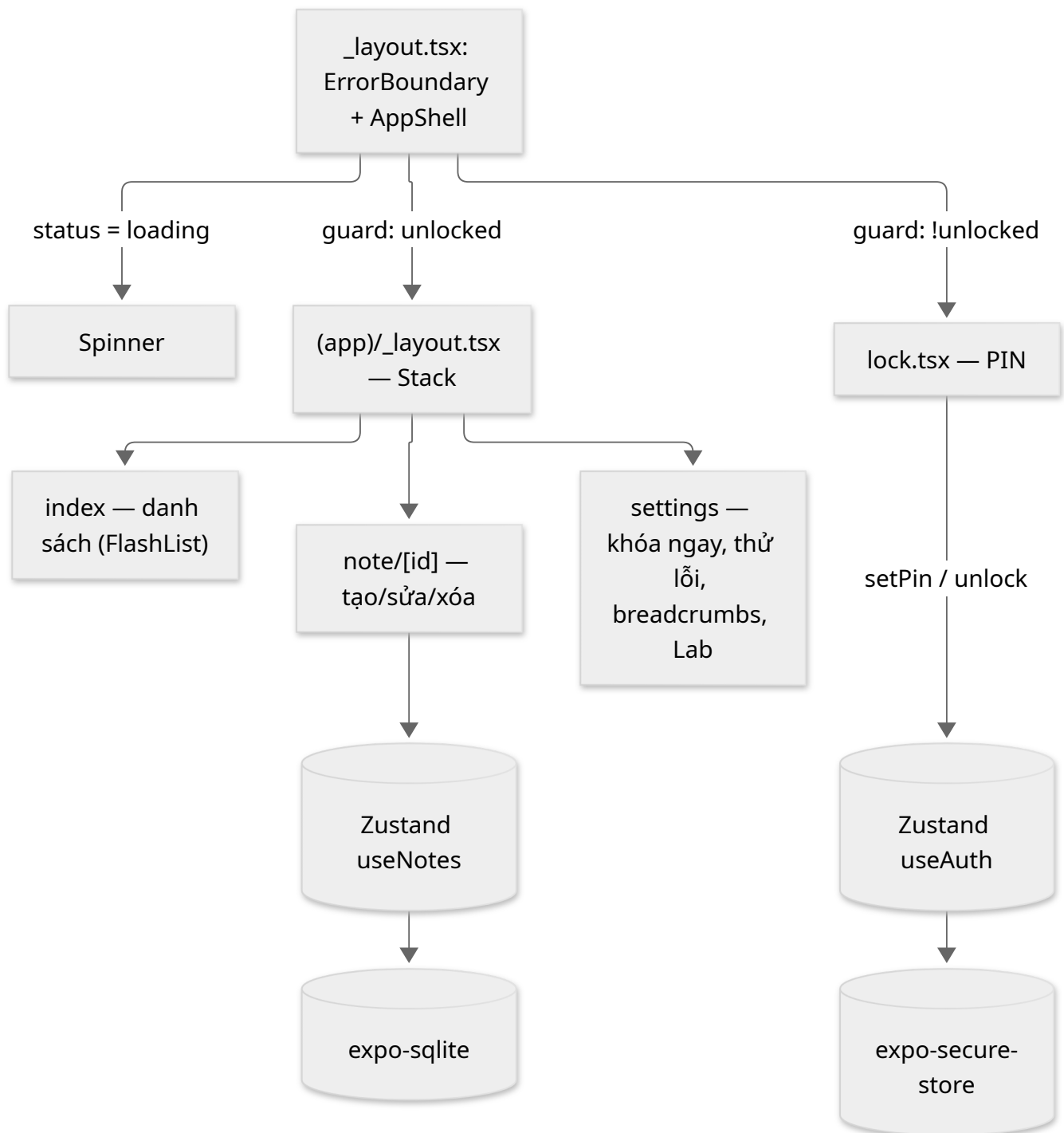
Chương 8 — App mẫu Tập 3: "Sổ Ghi Chú Bảo Mật"

Mục tiêu

Ghép Tập 3 thành một app:

- Màn hình **khóa bằng PIN** (tạo PIN lần đầu, mở khóa, khóa tạm khi sai nhiều), tự khóa khi app xuống nền.
- **Protected routes**: không vào được ghi chú khi chưa mở khóa, kể cả bằng deep link.
- Ghi chú lưu trong **SQLite**, danh sách dùng **FlashList v2**, **React Compiler** bật khi build.
- **Error Boundary** + logger có breadcrumbs; nút thử lỗi trong Cài đặt.
- Module native tự viết (`modules/text-stats`) với dự phòng JS; cấu hình **EAS** và workflow CI mẫu.

Giải thích đơn giản



Ví dụ

Chạy app (lệnh chính xác)

```
cd books/react-native/vol3-nang-cao/examples
npm ci
npx expo login          # cùng tài khoản với Expo Go
npx expo start          # quét QR bằng Camera
```

Trong Expo Go: module `text-stats` dùng bản JS (không có native); Face ID không có — app dùng PIN. **Chạy trên Expo Go: NOT RUN (không có iPhone trong sandbox).**

Kiểm tra:

```
npm run typecheck && npm run lint && npm test  
npm run export:ios
```

Kết quả kiểm tra toàn bộ Tập 3 (chạy thật 2026-09-28)

```
bash books/react-native/scripts/check-all.sh vol3-nang-cao
```

```
--- yaml (ci/*.yaml)  
yaml OK: 2 file  
--- typecheck  
--- lint  
--- test  
Test Suites: 10 passed, 10 total  
Tests:       45 passed, 45 total  
--- export web  
web bundle OK  
--- export ios  
ios bundle OK  
ALL CHECKS PASSED: vol3-nang-cao
```

Test toàn app (`src/__tests__/app.test.tsx`) — chạy mọi lớp thật (router, Zustand, SQLite qua `node:sqlite`, logic PIN); chỉ `SecureStore/Crypto` là bản giả:

```
PASS src/__tests__/app.test.tsx  
App Tập 3: Sổ Ghi Chú Bảo Mật  
  ✓ lần đầu: tạo PIN → vào danh sách ghi chú  
  ✓ PIN yếu bị từ chối  
  ✓ thêm, sửa, xóa ghi chú (lưu trong SQLite)  
  ✓ khóa lại → nhập sai → nhập đúng  
  ✓ deep link vào ghi chú khi đang khóa → chỉ thấy màn hình khóa (Protected route)  
  ✓ Error Boundary bắt lỗi từ nút "Thử lỗi" trong Cài đặt
```

Màn hình khóa (trích `src/app/lock.tsx`)

```
const submit = async () => {
  setError(null);
  if (creating) {
    if (pin !== confirm) return setError('Hai lần nhập PIN không khớp');
    const err = await setPin(pin);
    if (err) setError(err);
  } else {
    const ok = await unlock(pin);
    if (!ok) setError('PIN không đúng');
  }
  setPinText('');
  setConfirm('');
};
```

`TextInput` dùng `keyboardType="number-pad"`, `secureTextEntry`, `maxLength={6}`; khi đang bị khóa tạm, ô nhập bị vô hiệu và hiện "Thử lại sau N giây".

Store ghi chú (trích `src/state/notes.ts`)

```
add: async (input) => {
  const repo = await getNotesRepository();
  const now = Date.now();
  const note = await repo.create(input, newId(now), now);
  set({ notes: [note, ...get().notes] });
  logger.addBreadcrumb('info', 'note.create', { length: input.body.length });
  return note;
},
```

`getNotesRepository()` mở `secure-notes.db` một lần và chạy migration; repository dùng câu SQL có tham số ? (chống SQL injection).

Đi sâu

Những gì chạy được trong Expo Go và những gì cần development build

Tính năng	Expo Go SDK 57	Ghi chú
expo-secure-store, expo-crypto, expo-sqlite	Có	<code>requireAuthentication</code> với sinh trắc học: không
FlashList v2	Có (@shopify/flash-list@2.0.2)	JS-only, New Architecture
React Compiler	Có (Babel của Expo)	Đã thấy trong bundle <code>expo export</code>
Module <code>text-stats</code> native	Không → dùng bản JS	Cần development build
Face ID	Không	Cần development build
SQLCipher (mã hóa DB)	Không	Cần development build

Giới hạn bảo mật (nói thẳng)

- Ghi chú **không mã hóa** trong SQLite (xem Chương 4). PIN chỉ chặn truy cập qua giao diện app.
- Hash PIN là SHA-256 một vòng — ổn khi nằm trong Keychain, **không** đủ nếu bị lấy ra ngoài.
- Không có sao lưu/khôi phục: quên PIN = phải xóa dữ liệu (chức năng `resetAll` có trong store nhưng chưa có nút trên giao diện — một bài tập tốt).

Cấu trúc thư mục

```
examples/  
  app.json           # bundleIdentifier/package, scheme, experiments.reactCompiler  
  eas.json           # development / preview / production  
  metro.config.js    # wasm cho expo-sqlite trên web  
  jest.setup.js      # giả lập đo layout cho FlashList  
  modules/text-stats/  
    src/app/         # _layout (ErrorBoundary + Protected), lock, (app)/...  
    src/security/    # pin.ts (thuần), secureStorage.ts, autoLock.ts  
    src/notes/       # model, repository (SQL), db  
    src/state/       # useAuth, useNotes (Zustand)  
    src/monitoring/  # logger, ErrorBoundary, dedupe  
    src/chapters/chNN/  
    src/__tests__/   # test toàn app  
  ../ci/             # workflow GitHub Actions mẫu (book-checks.yml, eas-build.yml)
```

Lỗi và bẫy thường gặp

- **Protected route không có màn hình dự phòng** → người dùng bị kẹt; luôn có `lock` ở nhánh `!unlocked`.
- **Quên khóa khi xuống nền** → người khác cầm máy thấy ghi chú trong app switcher.
- **Mở DB nhiều lần** → dùng một promise dùng chung (`getNotesRepository`).
- `router.back()` **khi mở bằng deep link** → dùng `goBackOr` (từ Tập 1).
- **Test dùng chung DB** → `resetNotesRepositoryForTests()` trong `beforeEach`.
- **FlashList jestSetup của 2.0.2** làm FlashList undefined → dùng `jest.setup.js` của sách.

Tóm tắt

- App kết hợp: PIN + SecureStore, Protected routes, SQLite, FlashList, React Compiler, Error Boundary, logger.
- 45 test, gồm luồng tạo PIN → ghi chú → khóa/mở → deep link bị chặn → lỗi được bắt.
- Những phần cần tài khoản/phần cứng (build, submit, native, Face ID) được ghi rõ NOT RUN.

Bài tập (có lời giải)

Bài 1. Tự khóa khi không dùng app quá 1 phút (kể cả khi app vẫn mở). Viết hàm thuần và test.

▼ Lời giải

examples/src/security/autoLock.ts:

```
export function shouldAutoLock(lastActiveAt: number, now: number, timeoutMs: number): boolean {
  return now - lastActiveAt >= timeoutMs;
}
```

Test: `shouldAutoLock(0, 59_999, 60_000) === false`, `shouldAutoLock(0, 60_000, 60_000) === true`.

Gắn vào app: lưu `lastActiveAt` khi có chạm (`onTouchStart` ở View gốc) và kiểm tra bằng `setInterval` mỗi 10 giây, gọi `lockNow()` khi hàm trả `true`. (Phần gắn vào giao diện là lời giải tham khảo.)

Bài 2. Thêm nút "Quên PIN — xóa toàn bộ dữ liệu" ở màn hình khóa. Cần xóa những gì?

▼ Lời giải

1. `useAuth.getState().resetAll()` — xóa hash, salt, lock trong `SecureStore` → `status = 'no-pin'`.
2. Xóa ghi chú: `DELETE FROM notes` qua repository (thêm hàm `clearAll`), rồi `useNotes.getState().reset()`.
3. Hỏi xác nhận bằng `Alert` 2 bước (Tập 2 Chương 6 có cách test `Alert`).

Thứ tự quan trọng: xóa dữ liệu **trước**, rồi mới cho tạo PIN mới — tránh người lạ tạo PIN mới và đọc ghi chú cũ. (Lời giải tham khảo; chưa có trong app mẫu.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo Router — Protected routes: <https://github.com/expo/expo/blob/main/docs/pages/router/advanced/protected.mdx>
- Expo — SecureStore (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/securestore.mdx>
- Expo — SQLite (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/sqlite.mdx>

- Expo — FlashList (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/flash-list.md>
- Expo — React Compiler: <https://github.com/expo/expo/blob/main/docs/pages/guides/react-compiler.md>
- React Native — AppState: <https://github.com/facebook/react-native-website/blob/main/docs/appstate.md>

GLOSSARY — Thuật ngữ React Native (Anh → Việt)

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular
React Native (RN)	Framework viết app iOS/Android bằng React; vẽ view native thật	Ionic/Capacitor (nhưng Ionic dùng WebView)
Expo	Framework cho RN: CLI, router; thư viện module native, dịch vụ build	Angular CLI + thư viện chính thức
Expo SDK	Bộ phiên bản Expo đi cùng một phiên bản RN (sách dùng SDK 57 ↔ RN 0.86)	Phiên bản Angular major
Expo Go	App trên App Store để chạy thử project mà không cần build	—
Development build	Bản app "Expo Go của riêng bạn", có native module tùy ý	—
EAS (Expo Application Services)	Dịch vụ cloud: build, submit, update	CI/CD dịch vụ
Metro	Bundler (công cụ đóng gói JS) của RN	esbuild/webpack trong Angular CLI
Hermes	JavaScript engine chạy trên điện thoại; V1 mặc định từ RN 0.84	V8 trong trình duyệt
Fast Refresh	Cập nhật app ngay khi lưu file, giữ state	HMR (<code>ng serve --hmr</code>)
New Architecture	Kiến trúc mới của RN: JSI + Fabric + TurboModules; bắt buộc từ RN 0.82	—
JSI (JavaScript Interface)	Lớp C++ cho JS gọi native trực tiếp, không qua "bridge"	—
Fabric	Hệ thống render mới của RN	Renderer
TurboModules	Native module thế hệ mới, tải lười (lazy)	—
Codegen	Sinh code C++/native từ type TypeScript	—
Component	Hàm trả về JSX	<code>@Component</code>
JSX	Cú pháp giống HTML viết trong TypeScript	Template
Props	Tham số truyền vào component (chỉ đọc)	<code>@Input()</code>
Callback prop	Hàm truyền xuống để con báo sự kiện lên	<code>@Output()</code>
State	Dữ liệu thay đổi được của component (<code>useState</code>)	<code>signal()</code> / field của class
Hook	Hàm <code>use...</code> dùng tính năng React trong component hàm	Lifecycle hooks + <code>inject()</code>
Effect (<code>useEffect</code>)	Side effect sau khi render, có hàm dọn dẹp	<code>ngOnInit</code> + <code>ngOnDestroy</code> , <code>effect()</code>
Memo (<code>useMemo</code> , <code>memo</code>)	Ghi nhớ kết quả/Component để tránh tính/render lại	<code>computed()</code> , <code>OnPush</code>
Context	Truyền giá trị xuống cây component không qua props	Dependency Injection
Controlled component	Input mà giá trị nằm trong state React	<code>[(ngModel)]</code> , <code>FormControl</code>
Pressable	Component bắt sự kiện chạm	<code>(click)</code> trên <code><button></code>

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular
FlatList / SectionList	Danh sách ảo hóa (virtualized), chỉ render dòng đang thấy	<code>cdk-virtual-scroll</code>
Virtualization	Chỉ render phần tử đang nằm trên màn hình	Virtual scroll
Flexbox	Hệ thống bố cục; RN mặc định <code>flexDirection: 'column'</code>	CSS Flexbox
StyleSheet	API tạo style cho component	SCSS của component
Safe area	Vùng không bị tai thỏ/thanh home che	—
Expo Router	Router theo file (file-based routing) cho Expo	Angular Router
Layout (<code>_layout.tsx</code>)	File định nghĩa navigator (Stack, Tabs) cho một thư mục	Component chứa <code><router-outlet></code>
Stack / Tabs	Kiểu điều hướng: chồng màn hình / thanh tab dưới	—
Deep link	Link mở thẳng vào một màn hình trong app	URL route
Accessibility (a11y)	Khả năng tiếp cận: VoiceOver, vai trò, nhãn	ARIA
jest-expo	Preset Jest của Expo	Karma/Jest preset Angular
RNTL	React Native Testing Library — test theo cách người dùng dùng app	TestBed + Testing Library
Continuous Native Generation (CNG)	Thư mục <code>ios/</code> và <code>android/</code> được sinh ra từ <code>app.json</code> , không sửa tay	—
Config plugin	Plugin sửa cấu hình native khi sinh <code>ios/ / android/</code>	Angular schematics (gần giống)